Confluent Cloud Terraform Deployment — Scalar UDFs

> This example deploys **two scalar UDFs** (source in examples/scalar_udf/java/) to **Confluent Cloud** using Terraform: `CelsiusToFahrenheit` (converts Celsius to Fahrenheit) and `FahrenheitToCelsius` (converts Fahrenheit to Celsius). Both UDFs are stateless, deterministic, one-row-in / one-value-out functions, ship in the same uber JAR, and run as two independent streaming Flink statements in the same compute pool. All infrastructure (environment, Kafka cluster, Flink compute pool, service account, API keys, topics) and Flink SQL statements are declared as Terraform resources.

Table of Contents

- **1.0 Overview**
  - **1.1 How this differs from the other deployment paths**
- **2.0 How it works**
  - **2.1 Infrastructure provisioning**
  - **2.2 JAR delivery**
  - **2.3 Statement flow**
- **3.0 Prerequisites**
- **4.0 How to run**
  - **4.1 Deploy**
  - **4.2 Monitor**
  - **4.3 Tear down**
- **5.0 Resources**

1.0 Overview

1.1 How this differs from the other deployment paths

Aspect	**Confluent Cloud Terraform** (this example)	**Confluent Platform SQL Client**
Where it runs	Confluent Cloud	Confluent Platform on Minikube
How SQL is submitted	`confluent_flink_statement` Terraform resources	`sql-client.sh -f` on the JobManager pod
UDF JAR delivery	`confluent_flink_artifact` (uploaded to CC)	`kubectl exec` to Flink pods
Entry point	`make deploy-cc-scalar-udf`	`make deploy-cp-scalar-udf`
Requires code compilation	✅ (Java + Gradle for UDF JAR)	✅ (Java + Gradle for UDF JAR)
Statement lifecycle	Managed by Terraform state	Managed by Flink session cluster
Same codebase for both?	✅ (**same Java UDF code**, different Terraform vs SQL Client for deployment)	✅ (**same Java UDF code**, different Terraform vs SQL Client

Aspect	Confluent Cloud Terraform (this example)	Confluent Platform SQL Client
		for deployment)

2.0 How it works

2.1 Infrastructure provisioning

Terraform declares the full Confluent Cloud stack:

Resource	Purpose
`confluent_environment`	Isolated CC environment (`scalar-udf`) with Stream Governance Essentials
`confluent_kafka_cluster`	Standard single-zone Kafka cluster (AWS us-east-1)
`confluent_schema_registry_cluster` (data source)	Stream Governance Essentials (auto-provisioned with environment)
`confluent_service_account`	Service account with FlinkDeveloper, ResourceOwner (topic, transactional-id, schema-registry subject), and Assigner roles
`confluent_kafka_topic` × 4	`celsius_reading`, `celsius_to_fahrenheit`, `fahrenheit_reading`, `fahrenheit_to_celsius`
`confluent_flink_compute_pool`	Flink compute pool (max 10 CFU)
`kafka_api_key_rotation` module	Rotating API key pair for the Kafka service account
`flink_api_key_rotation` module	Rotating API key pair for the Flink service account

2.2 JAR delivery

On Confluent Cloud, UDF JARs are uploaded as **Flink artifacts** via the `confluent_flink_artifact` resource. The JAR is built from examples/scalar_udf/java/ and uploaded directly from the local build output (`app/build/libs/app-1.0.0-SNAPSHOT.jar`). Each `CREATE FUNCTION ... USING JAR` statement references the artifact using a `confluent-artifact://` URI.

2.3 Statement flow

Terraform manages the SQL statements for **both** scalar UDFs (`CelsiusToFahrenheit` and `FahrenheitToCelsius`) as `confluent_flink_statement` resources with explicit `depends_on` ordering. Both UDFs are bundled in the **same uber JAR**, so a single `confluent_flink_artifact` upload (Step 11) feeds both `CREATE FUNCTION` statements.

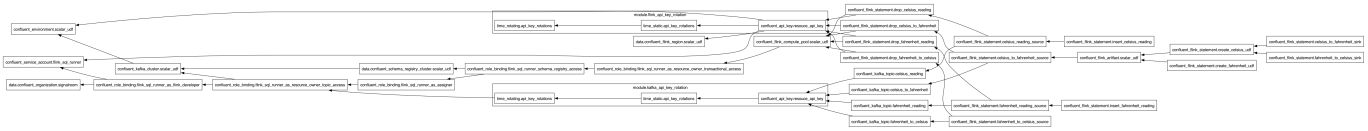
```
┌──────────────────────────────────────────────────────────────────────────────
│
│  ── UDF 1: CelsiusToFahrenheit ──
│
│  Step  1:  DROP TABLE IF EXISTS celsius_reading                → OK
```

```
│
│  Step  2:  CREATE TABLE celsius_reading (...)                → OK
│
│  Step  3:  INSERT INTO celsius_reading VALUES (sample data)   →
submitted│
│  Step  4:  DROP TABLE IF EXISTS celsius_to_fahrenheit         → OK
│
│  Step  5:  CREATE TABLE celsius_to_fahrenheit (...)           → OK
│
│
│
│  ── UDF 2: FahrenheitToCelsius ──
│
│  Step  6:  DROP TABLE IF EXISTS fahrenheit_reading            → OK
│
│  Step  7:  CREATE TABLE fahrenheit_reading (...)              → OK
│
│  Step  8:  INSERT INTO fahrenheit_reading VALUES (sample data)→
submitted│
│  Step  9:  DROP TABLE IF EXISTS fahrenheit_to_celsius         → OK
│
│  Step 10:  CREATE TABLE fahrenheit_to_celsius (...)           → OK
│
│
│
│  ── Shared artifact + both pipelines ──
│
│  Step 11:  Upload UDF JAR as confluent_flink_artifact         → OK
│
│            (contains BOTH scalar_udf.CelsiusToFahrenheit and
│
│             scalar_udf.FahrenheitToCelsius)
│
│  Step 12:  CREATE FUNCTION celsius_to_fahrenheit              → OK
│
│            AS 'scalar_udf.CelsiusToFahrenheit'
│
│            USING JAR 'confluent-artifact://...'
│
│  Step 13:  CREATE FUNCTION fahrenheit_to_celsius              → OK
│
│            AS 'scalar_udf.FahrenheitToCelsius'
│
│            USING JAR 'confluent-artifact://...'
│
│  Step 14:  INSERT INTO celsius_to_fahrenheit                  →
submitted│
│            SELECT sensor_id, celsius_temperature,
│
│                   celsius_to_fahrenheit(celsius_temperature)
│
│            FROM celsius_reading
│
│  Step 15:  INSERT INTO fahrenheit_to_celsius                  →
```

```
submitted│
│            SELECT sensor_id, fahrenheit_temperature,
│
│                   fahrenheit_to_celsius(fahrenheit_temperature)
│
│            FROM fahrenheit_reading
│
└──────────────────────────────────────────────────────────────────────────────
┘
```

Steps 14 and 15 are **two streaming jobs**, both running side-by-side from the same Flink compute pool: Step 14 reads from `celsius_reading` and writes the original Celsius value plus the converted Fahrenheit value to `celsius_to_fahrenheit`; Step 15 reads from `fahrenheit_reading` and writes the original Fahrenheit value plus the converted Celsius value to `fahrenheit_to_celsius`. Because scalar UDFs are one-row-in / one-value-out, each output row corresponds 1:1 with an input row.

Once deployment completes, Terraform generates a visual **resource graph** at `examples/scalar_udf/cc_deploy/terraform.png`, providing an at-a-glance view of the infrastructure and resource dependencies:



3.0 Prerequisites

- macOS with Homebrew or Linux with apt-get
- Java 21 and Gradle installed (for building the UDF JAR)
- Terraform installed
- A Confluent Cloud account with a **Cloud API key** and **secret** (create one here)
- The UDF JAR must be built before deploying:

```
make build-scalar-udf   # builds the UDF uber JAR from
examples/scalar_udf/java/
```

4.0 How to run

All commands are run from the **project root** (where the `Makefile` lives).

4.1 Deploy

A single target builds the UDF JAR and runs `terraform apply`:

```
make deploy-cc-scalar-udf CONFLUENT_API_KEY=<your-key>
CONFLUENT_API_SECRET=<your-secret>
```

Behind the scenes this runs:

Step	What it does
1	`./gradlew clean shadowJar` — builds the UDF uber JAR from `examples/scalar_udf/java/`
2	`terraform init` — initializes the Terraform working directory
3	`terraform apply -auto-approve` — provisions all CC infrastructure and submits Flink SQL statements
4	Generates a Terraform visualization at `examples/scalar_udf/cc_deploy/terraform.png`

4.2 Monitor

Monitor the running Flink statements in the Confluent Cloud Console:

1. Navigate to your **scalar-udf** environment
2. Open the **Flink** tab to see the compute pool and the **two streaming statements** (the `INSERT INTO celsius_to_fahrenheit ...` and the `INSERT INTO fahrenheit_to_celsius ...`)
3. Open the **Topics** tab to inspect all four topics: `celsius_reading`, `celsius_to_fahrenheit`, `fahrenheit_reading`, and `fahrenheit_to_celsius`

4.3 Tear down

To destroy all Confluent Cloud resources created by Terraform:

```
make teardown-cc-scalar-udf CONFLUENT_API_KEY=<your-key>
CONFLUENT_API_SECRET=<your-secret>
```

This runs `terraform destroy -auto-approve`, removing all Flink statements, the compute pool, Kafka topics, Kafka cluster, service account, and the environment.

5.0 Resources

- Confluent Terraform Provider
- confluent_flink_statement Resource
- confluent_flink_artifact Resource
- Create a User-Defined Function with Confluent Cloud for Apache Flink
- Flink Scalar Functions